Module IV – Data- and Model-Driven Machine Learning Approaches to Pharmacometrics

Instructor: Mohammad Kohandel
Students: Saranya Varakunan and Morghan van Walsum

In the previous modules, we introduced the mathematical foundations of pharmacometrics, including pharmacokinetic and pharmacodynamic models, pharmacometrics and population level analysis, parameter estimation, uncertainty quantification, and quantitative systems pharmacology. These approaches are primarily *mechanistic*, relying on differential equations and biological knowledge to describe drug behavior and treatment response.

Recent advances in machine learning have provided complementary approaches that can learn about drug behaviour and patient response directly from data. Broadly, these methods may be divided into two categories. **Data-driven approaches** use neural networks and other machine learning models to learn relationships directly from observed data with minimal assumptions about the underlying biological mechanisms. In contrast, **model-driven (or physics-informed) approaches** explicitly incorporate known information about pharmacological or biological dynamics into the learning process, combining the flexibility of neural networks with the interpretability and prior knowledge provided by mechanistic models.

This module is divided into two parts. We first examine data-driven approaches, including deep learning methods for pharmacometric prediction and Neural Ordinary Differential Equations (Neural ODEs), which learn continuous-time dynamics directly from data. We then introduce model-driven approaches, including Physics-Informed Neural Networks (PINNs), Physics-Informed Kolmogorov–Arnold Networks (PIKANs), and Systems Biology Informed Neural Networks (SBINNs), which embed differential equations and systems biology models into neural network training.

1 Data-Driven Approaches

1.1 Deep Learning for NCA

As discussed in Module 1, noncompartmental analysis (NCA) is one of the most widely used approaches for PK analysis. Unlike compartmental modeling, NCA is model-independent and estimates PK quantities directly from concentration–time data. Common NCA outputs include the maximum concentration ($C_{\max}$), the area under the concentration–time curve (AUC), and the elimination half-life ($t_{1/2}$). These quantities are routinely used in bioequivalence studies, drug–drug interaction studies, and dose selection.

Traditional NCA methods generally require relatively dense PK sampling in order to accurately estimate these quantities. When only sparse concentration measurements are available, numerical procedures such as trapezoidal integration and terminal slope estimation may become unreliable. To address this limitation, Liu et al [3] proposed Deep-NCA, a deep learning framework that learns to predict NCA parameters directly from sparse concentration–time measurements.

Noncompartmental Analysis as a Regression Problem

Deep-NCA formulates NCA estimation as a supervised regression problem. In machine learning, **regression** refers to the prediction of *continuous numerical quantities*. Given an input vector x and target value y, a regression model seeks to learn a function

$$f_\theta(x) \approx y,$$

where θ denotes the collection of trainable network parameters, such as the weights and biases of the neural network. During training, these parameters are iteratively adjusted using examples of known inputs and outputs so that the network learns to accurately map input data x to the desired target values.

Regression problems differ from **classification** problems, in which the output belongs to a *finite set of categories*. For example,

$$f_\theta(x) \in \{\text{Responder}, \text{Non-responder}\}$$

would represent a classification task, whereas

$$f_\theta(x) = \widehat{AUC}$$

is a regression task because the output is a continuous numerical quantity.

For Deep-NCA, the input consists of sparse PK time-series measurements,

$$x = \{(t_1, C_1), (t_2, C_2), \ldots, (t_n, C_n)\},$$

where t_i denotes a sampling time and C_i denotes the corresponding drug concentration. The outputs are three continuous NCA quantities,

$$y = \left(t_{1/2}, AUC_{\text{last}}, C_{\max}\right).$$

The learning objective is therefore to use regression to approximate the mapping

$$\{t_i, C_i\} \longrightarrow \left(t_{1/2}, AUC_{\text{last}}, C_{\max}\right),$$

allowing the network to predict clinically relevant PK quantities at unobserved time points directly from sparse concentration–time data. During training, the network parameters θ are adjusted so that the predicted outputs closely match the corresponding NCA values computed from dense PK profiles.

Generating Training Data

A major challenge in applying deep learning to pharmacometrics is the limited availability of large labeled datasets. To address this issue, Deep-NCA can be trained entirely on dense synthetic PK simulations before being applied to sparse experimental data. In the model developed by Liu et al, approximately one million PK profiles were generated for each drug class using standard two-compartment PK models with parameter ranges chosen to reflect realistic pharmacokinetic behavior.

The first step in generating training data is to determine the PK parameter range (P_{low}, P_{high}) based on their typical values, as determined by available population PK models (or other literature if you're brave). The bounds of this range may be calculated as

$$P_{low} = \alpha \times P_{95}$$
$$P_{high} = \beta \times P_{95}$$
where
$$P_{95} = \theta \times (\pm 1.96 \times \sqrt{\omega^2})$$

Here P_{95} is the 95 percentile interval of the individual PK parameter derived from the PPK model given the typical value of the parameter (θ) and the interindividual variability of the parameter (ω). P_{low} and P_{high} are the lower and upper boundaries of the PK parameter, respectively, where $\alpha \leq 1$ and $\beta \geq 1$ are extension factors to cover extreme PK profiles within the parameter boundaries.

For each simulated patient, dense concentration–time profiles can then be generated,

$$C(t) = f(t; \phi),$$

where ϕ denotes the underlying patient specific PK parameter vector, sampled from the ranges calculated above. Ground-truth NCA quantities for the synthetic data$\big(t_{1/2}, AUC_{\text{last}}, C_{\max}\big)$ can then be computed from the dense PK profiles using traditional NCA software

To mimic clinical trial conditions, sparse PK sampling schedules can be subsequently generated by selecting only a subset of the concentration measurements. The resulting dataset consisted of pairs

$$(x_i, y_i),$$

where x_i is a vector of sparse concentration–time measurements and y_i is a vector the corresponding NCA parameters computed from the full dense profile. x values can be chosen to capture key aspects of the PK profile such as absorption, maximum concentration, and distribution, thus enabling a more thorough characterization of the PK curve. The learning task is then to recover the dense-profile NCA estimates from sparse observations.

An important innovation of Deep-NCA is the use of **patient-specific normalization**. Given the heterogeneous nature of pharmacokinetic data, it is important for the Deep-NCA model to obtain invariant numeric results regardless of the units of the input data. For patient i, quantities such as concentration, dose, and time can be scaled by their patient-specific maxima,

$$\hat{C} = \frac{C}{C_{\max}}, \qquad \hat{t} = \frac{t}{t_{\max}},$$

producing dimensionless inputs between 0 and 1. This normalization improves training stability and allows the model to generalize across different concentration and dose scales, and across diverse datasets.

Convolutional Recurrent Neural Networks. The Deep-NCA model is based on a Convolutional Recurrent Neural Network (CRNN), which combines convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

Convolutional neural networks are designed to extract important local features from sequential data. Unlike a fully connected layer, which treats each input independently, a convolutional layer applies the same set of learnable weights across different regions of the input sequence. This process, known as *weight sharing*, allows the network to detect common patterns regardless of where they occur in the sequence while significantly reducing the number of trainable parameters.

Consider a one-dimensional input sequence

$$x = (x_1, x_2, \ldots, x_n),$$

and a convolutional filter (or kernel)

$$w = (w_1, w_2, \ldots, w_m),$$

of length m. The convolution operation computes a new sequence of features,

$$z_i = \sum_{k=1}^{m} w_k x_{i-k+1} + b,$$

where b is a bias term. The resulting quantity z_i is often referred to as a *feature map*, since it measures how strongly the local neighborhood surrounding position i resembles the pattern encoded by the filter weights. A nonlinear activation function $\sigma(\cdot)$ is then applied:

$$h_i = \sigma(z_i).$$

In practice, many filters are learned simultaneously. If F filters are used, then each filter produces its own feature map,

$$h_i^{(j)} = \sigma\left(\sum_{k=1}^{m} w_k^{(j)} x_{i-k+1} + b^{(j)}\right), \qquad j = 1, \ldots, F.$$

During training, the filter weights are adjusted so that the extracted features become useful for the prediction task. Rather than manually specifying which characteristics of the PK profile are important, the network automatically learns relevant patterns directly from the training data.

For pharmacokinetic time-series data, convolutional layers are particularly useful because many NCA quantities depend on local structure in the concentration–time profile. For example, neighboring measurements determine local slopes,

$$\frac{C_{i+1} - C_i}{t_{i+1} - t_i},$$

which are used in estimating elimination rates and identifying terminal phases. Similarly, local groups of concentration measurements determine the trapezoidal approximations used to estimate area under the curve,

$$AUC \approx \sum_i \frac{C_i + C_{i+1}}{2}(t_{i+1} - t_i).$$

Because convolutional filters operate on small local regions of the sequence, they are naturally suited for learning features related to concentration peaks, absorption phases, elimination phases, and local changes in exposure. These learned features are then passed to subsequent recurrent layers, which integrate information across the entire concentration–time profile.

However, many pharmacokinetic quantities depend not only on local patterns but also on relationships spanning the entire concentration–time profile. For example, estimation of $C_{\max}$ may depend primarily on a small neighborhood around the concentration peak, whereas quantities such as AUC_{last} and the elimination half-life $t_{1/2}$ depend on information accumulated across many sampling times. Consequently, the model must be able to retain information from earlier observations while processing later measurements.

Recurrent Neural Networks (RNNs) address this problem by maintaining an internal *hidden state* that evolves as the sequence is processed. At each time point t, the hidden state is updated according to

$$h_t = f(W_h h_{t-1} + W_x x_t + b),$$

where x_t denotes the current input, h_{t-1} denotes the previous hidden state, W_h and W_x are trainable weight matrices, b is a bias vector, and f is a nonlinear activation function. The hidden state therefore acts as a memory that summarizes information from all previous observations,

$$h_t = H(x_1, x_2, \ldots, x_t),$$

allowing later predictions to depend on the entire concentration–time sequence rather than only on local measurements.

Although standard RNNs are conceptually appealing, they often suffer from the *vanishing gradient problem*. During training, information from early time points must be propagated through many sequential updates. As the network becomes deeper in time, gradients may become extremely small, making it difficult for the model to learn long-range dependencies. Consequently, traditional RNNs often struggle to retain information from measurements occurring far apart in time.

To overcome this limitation, Deep-NCA uses **Gated Recurrent Units (GRUs)**. GRUs introduce additional gating mechanisms that regulate the flow of information through the network. In particular, the update gate determines how much information from the previous hidden state should be retained, while the reset gate determines how strongly previous information should influence the current update. A simplified GRU formulation is

$$z_t = \sigma(W_z x_t + U_z h_{t-1}),$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1}),$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t,$$

where z_t is the update gate, r_t is the reset gate, $\odot$ denotes element-wise multiplication, and σ is the logistic sigmoid function.

These gating mechanisms allow the network to selectively preserve important information while discarding irrelevant information. As a result, GRUs can learn dependencies over much longer time horizons than traditional RNNs.

For pharmacokinetic time-series data, this capability is particularly important because measurements obtained early in the profile may strongly influence quantities computed much later. For example, estimation of overall exposure depends on concentration values throughout the entire sampling period, while estimation of half-life requires the model to recognize relationships between early concentrations and the terminal elimination phase. GRUs therefore provide a natural mechanism for integrating information across the full concentration–time profile, complementing the convolutional layers that focus on local features.

1.1.1 Deep-NCA Architecture

The Deep-NCA architecture consists of six convolutional layers followed by two GRU layers and a final fully connected prediction layer. The convolutional layers extract local temporal features

from the PK profile, while the GRU layers aggregate information across the entire sequence. Finally, a fully connected layer maps the learned representation to the three target outputs

$$\hat{y} = \left(\widehat{t_{1/2}}, \widehat{AUC}_{\text{last}}, \widehat{C_{\max}}\right).$$

Conceptually, the architecture may be summarized as

$$\text{PK data} \rightarrow \text{CNN} \rightarrow \text{GRU} \rightarrow \text{Fully Connected Layer} \rightarrow \text{NCA Parameters}.$$

The motivation for this design is closely related to the structure of NCA itself. Convolutional layers are well suited for local computations, such as identifying concentration peaks or estimating local areas under the curve. Recurrent layers then combine these local features across the entire concentration–time profile to infer global PK quantities such as half-life and exposure.

1.1.2 Model Performance and Significance

The Deep-NCA model was evaluated on six previously unseen drugs, including both small and large molecules. Despite being trained entirely on simulated data, the model generalized well to all test compounds, achieving high coefficients of determination (R^2) for the prediction of half-life, AUC_{last}, and $C_{\max}$. For most drugs, R^2 values exceeded 0.9 and frequently approached 1.0.

The most significant result emerged when PK profiles were made increasingly sparse. Traditional NCA performance deteriorated substantially as sampling points were removed, particularly for half-life estimation. In contrast, Deep-NCA maintained relatively stable predictive performance even when several sampling times were omitted. Moreover, Deep-NCA was able to generate predictions for all simulated patients, whereas conventional NCA algorithms occasionally failed because terminal elimination phases could not be reliably identified.

This study illustrates how deep learning can complement traditional pharmacometric methodologies. Rather than replacing NCA, Deep-NCA learns to approximate NCA outputs from sparse observations, allowing investigators to obtain reliable PK summaries when conventional methods become unreliable. More broadly, the work demonstrates how machine learning can leverage large simulated datasets to solve practical pharmacometric problems while retaining clinically interpretable outputs.

Exercise 4.1: Deep-NCA and Sparse PK Sampling

Deep-NCA learns a mapping from sparse PK time-series data to continuous NCA outputs:

$$f_\theta : \{(t_j, C_j)\}_{j=1}^{m} \longmapsto \left(t_{1/2}, AUC_{\text{last}}, C_{\max}\right).$$

1. **Generate synthetic PK data.**

   Generate $N = 1000$ synthetic PK profiles from

   $$C(t) = C_0 e^{-kt}, \qquad C_0 \sim \text{Uniform}(50, 150), \qquad k \sim \text{Uniform}(0.05, 0.25).$$

   Compute the corresponding values of

   $$C_{\max}, \qquad AUC_{0,24}, \qquad t_{1/2},$$

   using dense sampling, then create sparse datasets by retaining only a subset of sampling times.

2. **Implement a CRNN.**

   Implement a small convolutional recurrent neural network consisting of

   - one 1-D convolutional layer,
   - one GRU layer,
   - one fully connected output layer,

   to predict

   $$(C_{\max}, AUC_{0,24}, t_{1/2})$$

   from sparse concentration–time profiles.

   Train the model using mean squared error loss and evaluate its performance using both MSE and R^2.

3. **Compare with another model.**

   Choose one alternative regression model (e.g., a multilayer perceptron, a one-dimensional CNN, a random forest, or gradient boosting) and train it on the same dataset.

   Compare its performance with the CRNN for sparse datasets containing

   $$m = 12,\ 8,\ 5,\ 3$$

   sampling points. Discuss which model is more robust as the sampling becomes increasingly sparse.

1.2 Neural ODEs for Pharmacology Modeling

Many pharmacometric models are dynamical systems. Drug concentrations, biomarker values, and treatment responses all evolve over time, and these processes are often modeled using

ordinary differential equations. A standard mechanistic model may be written as

$$\frac{dx}{dt} = f_{\text{mech}}(x, t, \theta),$$

where $x(t)$ denotes the biological or pharmacological state and θ denotes interpretable model parameters.

Traditional mechanistic models are useful because their parameters often have biological meaning. However, they can be difficult to construct when the system is complex, partially understood, or influenced by many patient-specific covariates. Neural networks, on the other hand, can learn flexible nonlinear relationships from high-dimensional data, but they do not naturally represent continuous-time biological dynamics. Neural Ordinary Differential Equations (Neural ODEs) aim to combine these two viewpoints by using a neural network to define the right-hand side of a differential equation.; while traditional ODEs describe the change in the state of a system using a function that depends on the current state and time, neural ODEs replace this function with a neural network to more effectively model complex non-linear dynamics.

1.2.1 Neural ODEs

To understand Neural ODEs, it is useful to first recall how information is processed in a standard deep neural network. A neural network consists of a sequence of layers, where the output of one layer becomes the input to the next. If h_k denotes the hidden state at layer k, then a traditional neural network computes a sequence of transformations

$$h_0 \to h_1 \to h_2 \to \cdots \to h_N.$$

The hidden state h_k can be thought of as an internal representation of the input data after it has passed through k layers of processing.

Many modern neural networks use a residual architecture, in which each layer learns only the change that should be applied to the current hidden state. Mathematically, this is written as

$$h_{k+1} = h_k + f(h_k, \theta_k),$$

where f is a neural network transformation and θ_k denotes the trainable parameters (weights and biases) associated with layer k. The quantity $f(h_k, \theta_k)$ can therefore be interpreted as the update applied to the hidden state.

This update rule closely resembles a numerical method used to solve ordinary differential equations. To see this, suppose the layers are separated by a small step size Δt. Then the update can be written as

$$h_{k+1} = h_k + \Delta t\, f(h_k, t_k, \theta).$$

Rearranging gives

$$\frac{h_{k+1} - h_k}{\Delta t} = f(h_k, t_k, \theta).$$

This expression is, usefully, the forward Euler approximation of an ordinary differential equation. Specifically, if

$$\frac{dh}{dt} = f(h, t, \theta),$$

then the Euler method approximates the solution using

$$h(t + \Delta t) \approx h(t) + \Delta t\, f(h(t), t, \theta).$$

The residual network update therefore behaves like a discretized dynamical system, where each layer corresponds to one numerical time step.

This observation motivates the central idea behind Neural ODEs. Rather than viewing a neural network as a finite sequence of discrete layers, we instead consider the limit as the layer spacing becomes arbitrarily small. In this way, the evolution of hidden states is treated as a continuous dynamic system. Taking $\Delta t \to 0$ yields the continuous-time model

$$\frac{dh(t)}{dt} = f_\theta(h(t), t),$$

where the unknown vector field f_θ is a neural network parameterize by θ that models the continuous transformation of the hidden state and replicates the ODE-dynamics. In other words, the neural network no longer computes a sequence of discrete transformations; instead, it learns the differential equation governing the evolution of the hidden state. The key idea here is that the Neural ODE is predicting a *derivative* of the hidden state, rather than the future hidden state itself. In this case, the derivative function is parameterized by a neural network.

Given an initial hidden state $h(t_0)$, the hidden state at a later time is obtained by solving

$$h(t) = h(t_0) + \int_{t_0}^{t} f_\theta(h(s), s)\, ds.$$

So, Neural ODEs replace the usual layer-by-layer propagation of information with the numerical solution of a learned differential equation. This makes neural ODEs **continuous depth models**, which have a number of characteristics that distinguish them from networks with a fixed number of discrete layers: (i) neural ODEs have an *adaptive depth*, allowing the network to adapt to the complexity of the input to achieve the desired accuracy; (ii) since neural ODEs can compute the hidden state at any time point without having to store all intermediate states, they have a *fixed memory cost*, allowing them to maintain memory efficiency while scaling with deep networks; (iii) neural ODEs provide a *smooth representation* of data, since the hidden states evolve continuously over time

This continuous-time formulation is particularly attractive in pharmacometrics because many biological and pharmacological processes are naturally modeled using differential equations. Furthermore, clinical measurements are often collected at irregular time intervals. Since the hidden state $h(t)$ is defined continuously in time, a Neural ODE can generate predictions at arbitrary time points without requiring observations to be interpolated onto a fixed time grid. This makes Neural ODEs especially well suited for longitudinal PK/PD and clinical trial datasets. However, the continuous-depth nature of neural ODEs presents a unique challenge in training the network; specifically, traditional backpropagation depends on tracking gradients through each intermediate layer, which is not possible when there are no layers. For smaller networks, the Neural ODE can be trained by treating each step of Euler integration as a network layer and proceeding with backpropagation as usual. However, for larger networks this method can be very energy intensive. The **Adjoint Sensitivity Method** addresses this challenge by providing a way to compute gradients by solving a second ODE backwards in time, but the technical aspects of this method are beyond the scope of this text.

1.2.2 Latent Neural ODEs

In clinical pharmacology, data are often sparse and irregularly sampled. Latent Neural ODEs are probabilistic models designed to account for this by approximating a latent representation that captures hidden or unobservable features in the input data. The main idea is to approximate a surrogate model for the underlying ODE by encoding the observed patient data into an initial latent state, evolving that latent state continuously in time, and then decoding it back into observable quantities.

Suppose patient i has longitudinal observations

$$\mathcal{D}_i = \{(t_{ij}, y_{ij})\}_{j=1}^{m_i},$$

where t_{ij} is the time of observation j, y_{ij} is the observed clinical or pharmacological measurement, and m_i is the number of observations for patient i.

Latent Neural ODE encoders are typically based on an RNN architecture. This architecture enables a global representation of the most important information and relevant patterns learned from the patient population. An encoder neural network summarizes this irregular time series into an initial latent state distribution:

$$q(z_{i0} \mid \mathcal{D}_i) = \mathcal{N}(\mu_i, \Sigma_i).$$

Here, μ_i and Σ_i are learned from the patient's observed data. A latent initial condition is then sampled from this distribution:

$$z_{i0} \sim q(z_{i0} \mid \mathcal{D}_i).$$

The latent state evolves according to a neural differential equation,

$$\frac{dz_i(t)}{dt} = f_\theta(z_i(t), t),$$

where f_θ is a neural network with trainable parameters θ. The latent trajectory is then obtained using an ODE solver:

$$z_i(t) = \text{ODESolve}\left(z_{t0}, f_\theta, t_0, ..., t_i\right).$$

Finally, a decoder maps the latent trajectory back to the observed clinical space:

$$\hat{y}_i(t) = g_\phi(z_i(t)),$$

where g_ϕ is a neural network decoder with parameters ϕ.

Putting these steps together, the latent Neural ODE workflow can be summarized as

$$\mathcal{D}_i \longrightarrow \text{encoder} \longrightarrow q(z_{i0} \mid \mathcal{D}_i) \longrightarrow z_{i0} \longrightarrow \text{ODE solver} \longrightarrow z_i(t) \longrightarrow \text{decoder} \longrightarrow \hat{y}_i(t).$$

This structure allows the model to interpolate between observed measurements and extrapolate beyond the observation window. In pharmacometric applications, this is particularly useful for predicting future PK concentrations, biomarker trajectories, or clinical outcomes from sparse early patient data.

1.2.3 Incorporating Dosing and Covariates

In pharmacometrics, patient trajectories are not determined only by time. They also depend on dosing history, treatment schedule, patient covariates, and previous observations. Therefore, the latent dynamics may be written more generally as

$$\frac{dz_i(t)}{dt} = f_\theta\big(z_i(t), t, d_i(t), c_i\big),$$

where $d_i(t)$ represents the dosing input for patient i and c_i represents patient-specific covariates such as body weight, renal function, biomarker status, or baseline disease burden.

The predicted PK concentration may then be written as

$$\hat{C}_i(t) = g_\phi(z_i(t)).$$

Similarly, a pharmacodynamic response may be predicted using

$$\hat{R}_i(t) = g_\phi^R(z_i(t)),$$

where $\hat{R}_i(t)$ could represent a biomarker level, tumor burden, adverse event risk, or another clinical endpoint.

This formulation allows Neural ODEs to learn patient-specific response trajectories while incorporating treatment information. For example, if the dosing regimen changes or treatment is stopped, this can be represented through a change in $d_i(t)$, and the Neural ODE can simulate the corresponding change in the predicted trajectory.

1.2.4 Connection to Classical PK/PD Models

Classical PK/PD models specify the right-hand side of the ODE using mechanistic assumptions. For example, a simple one-compartment PK model may be written as

$$\frac{dC}{dt} = -kC.$$

In this case, the structure of the dynamics is fixed and only the parameter k must be estimated.

In contrast, a Neural ODE replaces the fixed functional form with a learned function:

$$\frac{dC}{dt} = f_\theta(C, t).$$

This gives the model more flexibility, since f_θ can learn nonlinear dynamics directly from data.

A hybrid approach combines both ideas:

$$\frac{dx}{dt} = f_{\text{mech}}(x, t, \theta) + f_{\text{NN}}(x, t, \psi),$$

where f_{mech} represents known biological or pharmacological structure, while f_{NN} learns unknown or misspecified dynamics. This type of model is especially useful when part of the biology is well understood but some processes remain difficult to specify mechanistically.

For example, a PK model may be known, while the PD response is more complex. One could write

$$\frac{dC}{dt} = f_{\text{PK}}(C, t, \theta),$$

$$\frac{dR}{dt} = f_{\text{PD,NN}}(R, C, t, \psi),$$

where $C(t)$ is drug concentration and $R(t)$ is response. In this case, the PK component remains mechanistic, while the response dynamics are learned from data.

1.2.5 Applications in Clinical Pharmacology

Neural ODEs are particularly useful for longitudinal clinical datasets because they naturally handle irregularly sampled observations and represent patient trajectories in continuous time. In pharmacokinetic modeling, they may be used to predict concentration–time profiles, extrapolate future drug concentrations, and simulate alternative dosing regimens. In pharmacodynamic modeling, they can be used to describe biomarker dynamics, disease progression, and treatment response trajectories.

A major advantage of Neural ODEs is their ability to learn patient-specific dynamics from sparse observations. By encoding patient history into a latent state, they can generate individualized trajectories and incorporate high-dimensional covariates such as demographic, laboratory, imaging, and genomic data.

However, Neural ODEs also have limitations. Training requires repeatedly solving differential equations, which can be computationally expensive, and the learned dynamics may be difficult to interpret biologically. Also, performance may deteriorate when datasets are small or noisy. As such, careful validation and comparison with established PK/PD models remain essential.

Overall, Neural ODEs provide a flexible framework for learning unknown dynamics from data while preserving the continuous-time structure that is central to pharmacometric modeling.

Exercise 4.2

1. The encoder network for Latent Neural ODEs is typically an RNN. What features of RNNs make this architecture a logical choice?
2. Given the provided sample code for Neural ODEs, answer the following questions
   (a) What is the purpose of the *ODEFunc* and *ODESolver* classes?
   (b) How is the data used to help the network learn the function?
   (c) What is the relevance of the vector field plots?
   (d) For both the synthetic and real data examples, write some code to recover the symbolic form of the predicted ODE

2 Model-Driven Approaches

2.1 Universal Differential Equations

A **universal approximator** is a mathematical object that can accurately represent any continuous function, given sufficient depth or capacity. Low dimensional examples include method such as Fourier expansions, and higher dimensional examples include machine learning models such as neural networks. **Universal differential equations (UDEs)** are differential equations that are defined in part by a universal approximator, which can be used to approximate any unknown components of the differential equation. Neural ODEs are a special case of this framework in which the *entire* DE is unknown and defined by a neural network; in general, however, UDEs assume only *some* of the DE is unknown, which allows for the integration of well defined mechanistic knowledge and improved results given only sparse observations. Given observational data that follows some unknown, real-valued function u, the right hand side of the time derivative $\mathcal{N}$ may be formulated as a function g of known parameters θ and unknown

functions $h_1, ..., h_k$. That is,

$$\frac{\partial u}{\partial t} = \mathcal{N}[u; \theta]$$

where

$$\mathcal{N}[u; \theta] = g(u, h_1(u; \theta), ..., h_k(u; \theta); \theta)$$

The UDE framework was introduced by Rackauckas et al [6], who approximated the unknown functions by a single neural network H with k outputs. With any approximation of H, the time derivative of $\frac{\partial u}{\partial t}$ can be fully defined and u can be solved for numerically, yielding a neural network approximation of the DE solution $u_\theta(t)$. The loss function $\mathcal{L}$ can then be defined as the Euclidean distance or MSE between the predicted DE solution and the provided data. In this way, H can be progressively updated to better represent the data.

Recall that Euclidean loss at discrete data points (t_i, d_i) is given by

$$\mathcal{L}_{Euc} = \sum_i ||u_\theta(t_i) - d_i||$$

and MSE loss is given by

$$\mathcal{L}_{MSE} = \sum_i ||u_\theta(t_i) - d_i||^2$$

Using localized derivative-based methods such as gradient descent to optimize these loss functions requires the calculation of $\frac{d\mathcal{L}}{d\theta}$, which in turn requires the calculation of $\frac{du}{d\theta}$. As with Neural ODEs, these gradients can be efficiently calculated using the adjoint method. And, as with Neural ODEs, the technical details of this method are beyond the scope of this text.

2.2 Physics-Informed Neural Networks

Physics-informed neural networks (PINNs) are neural network models that are trained to fit observed data while also satisfying known differential equations. In standard supervised learning, a neural network is trained only by comparing its predictions with data. In a PINN, the governing equations are included directly in the loss function, so the network is encouraged to produce outputs that are both consistent with the observed data mechanistically plausible.

Consider a mechanistic model written as

$$\frac{dx}{dt} = f(x, \theta, t), \qquad x(0) = x_0,$$

where $x(t) \in \mathbb{R}^m$ denotes the state vector, θ denotes the model parameters (which may be known or unknown), and f represents the mechanistic dynamics. In a PINN, the state trajectory is approximated by a neural network

$$x(t) \approx \hat{x}(t; w),$$

where w denotes the trainable weights and biases of the network. The derivative $\frac{d\hat{x}}{dt}$ is computed implicitly as the neural network learns using automatic differentiation, and can be incorporated into the loss function to allow the network to evaluate how well the neural approximation satisfies the governing ODE.

Precisely, the physics residual is defined as

$$r(t) = \frac{d\hat{x}}{dt}(t; w) - f(\hat{x}(t; w), \theta, t).$$

If $\hat{x}(t; w)$ exactly satisfies the ODE, then $r(t) = 0$ for all t. During training, this residual is evaluated at a set of collocation points $\{\tau_j\}_{j=1}^{M}$, which are time points at which the derivative from the governing differential equation is calculated and compared to $\frac{d\hat{x}}{dt}$.

The total loss usually contains three main components:

$$L = \lambda_{\text{data}} L_{\text{data}} + \lambda_{\text{phys}} L_{\text{phys}} + \lambda_{\text{IC}} L_{\text{IC}},$$

where L_{data} measures agreement with observations, L_{phys} penalizes violations of the ODE, and L_{IC} enforces the initial condition. λ_{data}, λ_{phys}, and λ_{IC} are loss weights that control the relative importance of each term. Each component is typically calculated as:

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} \|y_i - \hat{x}(t_i; w)\|^2,$$

$$L_{\text{phys}} = \frac{1}{M} \sum_{j=1}^{M} \|r(\tau_j)\|^2,$$

and

$$L_{\text{IC}} = \|\hat{x}(0; w) - x_0\|^2.$$

Here, $\{(t_i, y_i)\}_{i=1}^{N}$ are observed data points.

This construction differs from a purely data-driven neural network because the model is not free to fit the data arbitrarily. Even in regions where no data are available, the physics residual encourages the predicted trajectory to remain consistent with the mechanistic model. This is especially useful in pharmacometrics, where measurements are often sparse, noisy, and collected at irregular time points.

PINNs can also be used for inverse problems, which seek to recover unknown function parameters from observed data. In this setting, the unknown parameters θ are trained jointly with the neural network weights w. The optimization problem becomes

$$(w^*, \theta^*) = \arg\min_{w,\theta} L(w, \theta).$$

Thus, the PINN simultaneously reconstructs the state trajectory and estimates unknown parameters. This is useful when the ODE structure is known but quantities such as clearance rates, transfer rates, growth rates, or drug-effect parameters must be inferred from data.

A key advantage of PINNs is that they can handle partial observation. Suppose the full state is $x(t) = (x_1(t), x_2(t), x_3(t))$, but only $x_2(t)$ is measured. The data loss may then be written as

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} \left|y_i - \hat{x}_2(t_i; w)\right|^2,$$

while the physics loss is still imposed on all components of the ODE system. In this way, the mechanistic equations help regularize unobserved states.

2.2.1 PINNs for PK/PD and QSP

PINNs are well suited to pharmacokinetic and pharmacodynamic modeling because many PK/PD and QSP models are formulated as systems of differential equations. In these applications, the states may represent drug amounts, plasma concentrations, biomarker levels, tumor burden, receptor occupancy, immune cell populations, or other biological variables.

For example, consider a one-compartment oral absorption model:

$$\frac{dA_g}{dt} = -k_a A_g,$$

$$\frac{dA_c}{dt} = k_a A_g - \frac{CL}{V} A_c,$$

$$C(t) = \frac{A_c(t)}{V},$$

where $A_g(t)$ is the amount of drug in the gastrointestinal compartment, $A_c(t)$ is the amount in the central compartment, k_a is the absorption rate, CL is clearance, V is volume of distribution, and $C(t)$ is plasma concentration.

A PINN approximation may take the form

$$(\hat{A}_g(t), \hat{A}_c(t)) = \mathcal{N}(t; w).$$

The residuals are then

$$r_g(t) = \frac{d\hat{A}_g}{dt} + k_a \hat{A}_g,$$

and

$$r_c(t) = \frac{d\hat{A}_c}{dt} - k_a \hat{A}_g + \frac{CL}{V} \hat{A}_c.$$

If concentration data C_i are observed at times t_i, the data loss can compare the measured concentration with the predicted concentration

$$\hat{C}(t_i) = \frac{\hat{A}_c(t_i)}{V}.$$

The total loss becomes

$$L = \lambda_{\text{data}} \frac{1}{N} \sum_{i=1}^{N} |C_i - \hat{C}(t_i)|^2 + \lambda_{\text{phys}} \frac{1}{M} \sum_{j=1}^{M} \left(|r_g(\tau_j)|^2 + |r_c(\tau_j)|^2 \right) + \lambda_{\text{IC}} L_{\text{IC}}.$$

If k_a, CL, or V are unknown, they can be included as trainable parameters.

PINNs can also be applied to pharmacodynamic models. For example, a tumor growth inhibition model may be written as

$$\frac{dT}{dt} = k_g T \left(1 - \frac{T}{T_{\max}} \right) - E(t) T,$$

where $T(t)$ denotes tumor burden, k_g is the tumor growth rate, $T_{\max}$ is a carrying capacity, and $E(t)$ represents drug-induced killing or treatment efficacy. If $E(t)$ is known, the PINN can estimate parameters such as k_g and $T_{\max}$. If $E(t)$ is unknown, the PINN can be extended to infer it.

This leads to a gray-box formulation. In gray-box modeling, part of the ODE system is trusted mechanistically, while another component is unknown and learned from data. For example, suppose the dynamics of a tumor are modeled by

$$\frac{dT}{dt} = k_g T \left(1 - \frac{T}{T_{\max}} \right) - \phi(t) T,$$

where $\phi(t)$ is an unknown time-varying treatment effect. A neural network can represent both the state and the unknown function:

$$(\hat{T}(t), \hat{\phi}(t)) = \mathcal{N}(t; w).$$

The residual becomes

$$r_T(t) = \frac{d\hat{T}}{dt} - k_g\hat{T}\left(1 - \frac{\hat{T}}{T_{\max}}\right) + \hat{\phi}(t)\hat{T}.$$

Training the PINN then allows the model to recover a hidden treatment-effect function while still respecting the assumed tumor-growth structure.

This type of formulation is useful in pharmacometrics and QSP because many biological processes are only partially known. For instance, the modeler may trust the compartment structure of a PK model but not know the exact form of nonlinear absorption. Similarly, in QSP, the known components may describe receptor binding, signaling, or cell population dynamics, while an unknown feedback term or resistance mechanism must be learned from data.

A general gray-box system can be written as

$$\frac{dx}{dt} = f_{\text{known}}(x, \theta, t) + f_{\text{NN}}(x, t; w),$$

where f_{known} represents the mechanistic component and f_{NN} represents the unknown component to be recovered by a neural network. The residual is

$$r(t) = \frac{d\hat{x}}{dt} - f_{\text{known}}(\hat{x}, \theta, t) - f_{\text{NN}}(\hat{x}, t; w).$$

This allows PINNs to serve not only as parameter estimators, but also as tools for discovering missing dynamical terms.

Several practical details are important in PK/PD applications. First, collocation points should cover the time intervals where important dynamics occur. Uniform collocation is a useful baseline, but adaptive or residual-based collocation may be helpful when trajectories contain sharp peaks, abrupt dose effects, or fast transients. Second, loss terms should be monitored separately; a small total loss can hide the fact that the data loss is low while the physics residual remains large, or vice versa. Third, constraints should be imposed when parameters have known biological ranges. For example, clearance, volume, rate constants, and efficacy parameters should be positive. This can be enforced using transformations such as

$$\theta = \log(1 + e^{\eta}),$$

where η is an unconstrained trainable variable and $\theta > 0$ is the physical parameter.

PINNs also require careful validation. A good fit to observed data is not sufficient. One should check whether the physics residual is small, whether recovered parameters are biologically plausible, whether hidden states or hidden functions are reasonable, and whether the inferred model can reproduce the trajectory using a conventional ODE solver. This last step is important: after estimating parameters or unknown functions with a PINN, one can solve the original ODE numerically using the recovered quantities and compare the result with the PINN trajectory and observed data.

Exercise 4.3: Phyics-Informed Neural Networks

Consider the one-compartment PK model

$$\frac{dC}{dt} = -kC, \qquad C(0) = C_0,$$

where $k > 0$ is the elimination rate constant.

1. **Physics-informed loss.**

   Suppose the concentration is approximated by a neural network

   $$\hat{C}(t; \theta).$$

   Derive expressions for the data loss L_{data}, the physics loss L_{phys}, and the initial-condition loss L_{IC}. Hence write the complete PINN loss function.

2. **Collocation points.**

   Suppose concentration measurements are available at only eight time points, but the PINN uses 200 collocation points.

   Explain why the number of collocation points may greatly exceed the number of observations. Discuss how increasing or decreasing the number of collocation points would affect training accuracy and computational cost.

3. **Implement a PINN.**

   Generate synthetic concentration data from the one-compartment model, randomly retain only 10 observation times, and implement a PINN to recover the complete concentration–time profile.

   Compare the PINN prediction with the analytical solution and report the mean squared error.

4. **Inverse parameter estimation.**

   Treat the elimination rate constant k as an unknown trainable parameter and modify your PINN so that it simultaneously estimates both $C(t)$ and k.

   Investigate how the accuracy of the recovered parameter changes as the number of observed concentration measurements is reduced.

2.2.2 Physics-Informed Kolmogorov–Arnold Networks

The most common implementation of a Physics-Informed Neural Network uses a **multilayer perceptron (MLP)** to approximate the unknown solution. As discussed previously, an MLP is built from layers of neurons connected by trainable weights. Each layer applies a linear transformation followed by a nonlinear activation function, such as tanh, before passing information to the next layer.

If the input is t, an L-layer MLP can be written as

$$z^{(0)} = t,$$

$$z^{(\ell)} = \sigma\left(W^{(\ell)} z^{(\ell-1)} + b^{(\ell)}\right), \qquad \ell = 1, \ldots, L,$$

followed by the output layer

$$\hat{x}(t) = W^{(L+1)} z^{(L)} + b^{(L+1)}.$$

Here, $W^{(\ell)}$ and $b^{(\ell)}$ are trainable weights and biases, while $\sigma(\cdot)$ is a nonlinear activation function. During training, these parameters are adjusted so that the network approximates the desired solution.

Physics-Informed Kolmogorov–Arnold Networks (PIKANs) use exactly the same physics-informed framework as PINNs. The differential equations, physics residuals, collocation points, and loss function are all unchanged. The only difference is the neural network architecture used to represent the unknown solution: Instead of using fixed activation functions such as tanh, a Kolmogorov–Arnold Network (KAN) learns a collection of **univariate functions**. A univariate function is simply a function of a single variable. For example,

$$f(x) = x^2, \qquad g(x) = \sin(x),$$

are both univariate functions because they each depend on only one input.

Rather than applying a fixed activation function at every neuron, a KAN learns these one-dimensional functions directly from the data. A simplified KAN layer can be written as

$$z_j^{(\ell)} = \sum_i \psi_{ij}^{(\ell)} \left(z_i^{(\ell-1)} \right),$$

where each $\psi_{ij}^{(\ell)}$ is a trainable univariate function.

In practice, these functions cannot be stored exactly and must themselves be represented mathematically. One common approach is to express each function as a weighted sum of **Chebyshev polynomials**,

$$\psi(\zeta) = \sum_{n=0}^{D} c_n T_n(\zeta),$$

where $T_n(\zeta)$ denotes the n-th Chebyshev polynomial and c_n are trainable coefficients.

Chebyshev polynomials are a family of orthogonal polynomials that provide an efficient way of approximating smooth functions. Just as a Fourier series approximates a function using sines and cosines, a Chebyshev expansion approximates a function using polynomial basis functions. During training, the coefficients c_n are learned, allowing the network to adapt the shape of each univariate function. Because KANs learn the nonlinear functions themselves rather than relying on fixed activation functions, they can provide a more flexible representation of complex nonlinear relationships. For this reason, PIKANs have recently been explored as an alternative to standard PINNs for inverse problems, parameter estimation, and gray-box model discovery in pharmacometrics and quantitative systems pharmacology.

A physics-informed KAN is trained using the same composite loss:

$$L = \lambda_{\text{data}} L_{\text{data}} + \lambda_{\text{phys}} L_{\text{phys}} + \lambda_{\text{IC}} L_{\text{IC}}.$$

A PINN and a PIKAN is have the same physics residual, collocation points, and mechanistic model. The only difference is the *neural function approximator* used to represent the state or unknown function.

PIKANs may be useful in inverse PK/PD and QSP problems because their learnable univariate functions can provide a different inductive bias from MLPs. In some settings, this may improve representation efficiency or robustness, especially when the unknown function is smooth but

difficult to approximate with a standard MLP. For example, if a time-varying efficacy function $\phi(t)$ must be inferred from sparse observations, a Chebyshev-based KAN may represent this function using structured polynomial components.

However, PIKANs are not automatically better than standard PINNs. They may be more computationally demanding, require more careful tuning, and be sensitive to the degree of the polynomial expansion. In practice, a reasonable workflow is to begin with an MLP-based PINN as a baseline and then compare with a PIKAN using the same data, residuals, collocation points, and loss weights. This makes it possible to determine whether performance differences are due to the neural representation rather than changes in the physics-informed formulation.

Overall, PINNs and PIKANs provide a flexible framework for combining mechanistic pharmacometric models with neural networks. They are particularly useful for inverse problems, partial observation, gray-box discovery, and time-varying treatment-effect inference. Their main strength is that they use the governing equations not only after model fitting, but throughout training, allowing biological and pharmacological structure to guide the learning process.

Exercise 4.4: Physics-Informed Kolmogorov–Arnold Networks

Consider the one-compartment PK model

$$\frac{dC}{dt} = -kC, \qquad C(0) = C_0,$$

approximated using a PIKAN.

1. **Chebyshev basis.**

   Using

   $$T_0(x) = 1, \qquad T_1(x) = x,$$

   and

   $$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x),$$

   derive $T_2(x)$, $T_3(x)$, and $T_4(x)$.

2. **Physics-informed loss.**

   Suppose

   $$\hat{C}(t) = \sum_{n=0}^{D} c_n T_n(t).$$

   Derive the physics residual

   $$r(t) = \frac{d\hat{C}}{dt} + k\hat{C},$$

   and write the complete PIKAN loss containing data, physics, and initial-condition terms.

3. **PINN versus PIKAN.**

   Implement both

   - an MLP-based PINN, and
   - a Chebyshev-based PIKAN,

   using the same training data, collocation points, and optimizer. Compare the prediction error, physics residual, and training time.

4. **Polynomial degree.**

   Repeat the PIKAN experiment for

   $$D = 2,\ 5,\ 10.$$

   How does the polynomial degree affect accuracy, convergence, and computational cost?

2.3 Systems Biology Informed Neural Networks

Variability and unpredictability in patient responses to drugs are common, often as a result of factors relating to patient-specific biology. **Systems biology models** provide mechanistic insight and can be used to generate an unlimited amount of meaningful synthetic clinical data, given appropriate experimental calibration. However, these models lack an integrated computational discovery approach; these models may be computationally expensive to calibrate and

have unknown parameter values. Machine Learning methods such as **Neural Networks** are powerful tools for analyzing and interpreting high-dimensional data sets, such as those that accompany experimental studies of biological systems, but they require a large amount of training data to effectively make accurate predictions that can be generalized to unseen data. Additionally, neural networks alone provide little biological interpretability or mechanistic insight. **Transfer learning** can be used to bridge this gap, using a systems biology model to generate simulated data that can then be used to pre-train machine learning models. Pretraining allows the model to learn biological relationships, rather than simply memorizing data points, allowing for more nuanced predictions when applied to clinical data. Integrating systems biology with machine learning also addresses the black box nature of machine learning models, allowing for the results of the machine learning model to be interpreted in a meaningful way. Below we provide an outline for the Systems Biology Informed Neural Network (SBINN) approach introduced by [5].

2.3.1 Systems Biology

Systems biology is the computational and mathematical analysis of biological systems. This field aims to provide a systems-level understanding of complex biological systems by providing a holistic and interpretable description of system structures and dynamics. Systems biology models often consist of coupled ODEs that describe the temporal dynamics and interactions between the populations of interest. Many biological and chemical pathways have known structural pathways from which ODE systems can be derived following kinetic laws. This model can then be integrated with experimental data to estimate the nominal parameters of the model using parameter fitting methods (genetic algorithm etc). Note that the parameter sets obtained using these methods may not necessarily be unique, as there are often many sets of parameters that can adequately fit the average patient data. To gain a better understanding of the mechanisms of drug action or other outcomes within the model, a sensitivity analysis may be conducted.

2.3.2 Simulated Clinical Trials

The calibrated systems biology model can be used to generate simulated clinical trials with virtual populations. The parameter ranges used to generate the virtual populations may be taken from existing data and literature, or estimated if necessary. The simulated data may then be analyzed to determine which patient features contribute most to variability in patient response, and in turn to identify potentially important resistance or reaction nodes. The simulated clinical trial data is additionally used to pretrain a neural network that can then be applied to clinical or experimental data to predict response phenotypes in a practical setting.

2.3.3 Feature Selection

When developing machine learning models, it is often important to determine which kinetic parameters and experimentally obtainable values are most important in distinguishing response phenotypes. The identification of influential features helps to identify potential mechanisms driving the outcome of interest, such as drug resistance., and can help develop more targeted experiments for future investigation. Performing feature selection on simulated data can also be used to illustrate the validity of simulated data in supplementing small data sets; feature selection from simulated data should produce classification results that are as accurate as those produced with feature selection from clinical data. When the feature space exceeds the number of samples, it is not appropriate to apply feature selection to the clinical data directly.

2.3.4 Neural Networks

Non-linear Regression. A regression neural network can be used to estimate the nominal parameter values of the patients included in the experimental dataset, after pre-training on simulated patient data. The virtual population is generated using the systems biology model, given alterations in initial conditions only; all other kinetic parameters, which are estimated to fit the average patient data, are kept constant. Methods for generating virtual populations, such as Latin Hypercube Sampling, can be used to generate statistically independent samples of initial conditions and kinetic parameter values. The correlation between kinetic parameters and measurable population values can be calculated to determine the best input values for the neural network, where populations with higher correlations to the parameter values are considered stronger inputs. After generating the simulated clinical trial it can be used to train the recursion NN, which enables the identification of important correlations between kinetic parameter values and experimentally measurable features. The hyperparameters of the network can then be tuned to achieve optimal convergence of the neural network outputs to the simulated patient data. After training, this network can be applied to experimental data to infer the kinetic parameters of patients given experimentally measurable inputs only.

Classification. The results of the previously described feature selection methods can be combined with a classification neural network to predict patient response phenotype. Patients can be randomly sampled from the simulated clinical trial to train this model; based on the available data, it may be beneficial to intentionally randomly sample an imbalanced data set to match the ratio of relevant responder phenotypes in the experimental data set. After training on the simulated patient data, transfer learning can be applied by retraining the pretrained neural network on a subset of the experimental data, again randomly sampled to approximately preserve the ratio of response phenotypes. The purpose of this additional training step is to improve the prediction accuracy on the experimental data.

References

[1] Nazanin Ahmadi, Shupeng Wang, and George Karniadakis. Pharmacometrics modeling via physics-informed neural networks: Integrating time-variant absorption rates and fractional calculus for enhancing prediction accuracy. *arXiv preprint arXiv:2412.21076*, 2024.

[2] Nazanin Ahmadi Daryakenari, Khemraj Shukla, and George Em Karniadakis. Representation meets optimization: Training pinns and pikans for gray-box discovery in systems pharmacology. *Computers in Biology and Medicine*, 201:111393, 2026.

[3] Guohua Liu, Lauren Brooks, Jennifer Canty, Dan Lu, Jin Y. Jin, and Jing Lu. Deep-nca: A deep learning methodology for performing noncompartmental analysis of pharmacokinetic data. *CPT: Pharmacometrics & Systems Pharmacology*, 13(5):870–879, 2024.

[4] Ignacio B. Losada and Nicola Terranova. Bridging pharmacology and neural networks: A deep dive into neural ordinary differential equations. *CPT: Pharmacometrics & Systems Pharmacology*, 13(8):1289–1296, 2024.

[5] M. Przedborski, M. Smalley, S. Thiyagarajan, S. Shrestha, C. J. Perry, P. Tsvetkov, et al. Systems biology informed neural networks (sbinn) predict response and novel combinations for pd-1 checkpoint blockade. *Communications Biology*, 4:877, 2021.

[6] Christopher Rackauckas, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. Universal differential equations for scientific machine learning, 2021.

[7] S. S. Sidharth, A. R. Keerthana, K. P. Anas, et al. Chebyshev polynomial-based kolmogorov–arnold networks: An efficient architecture for nonlinear function approximation. *arXiv preprint arXiv:2405.07200*, 2024.